

Proyecto de Grado: IA al servicio de las comunidades

Maestría en Analítica para la Inteligencia de Negocios

Nombres: María Fernanda Izquierdo Aparicio & Ana María Ochoa Muñoz

Fecha: 19 de Mayo de 2025

Este modelo forma parte del proyecto de grado de la Maestría en Analítica para la Inteligencia de Negocios de la Pontificia Universidad Javeriana, enfocado en la aplicación de Inteligencia Artificial al servicio de las comunidades. Su objetivo es clasificar imágenes de plantas en función de su especie usando una arquitectura pre-entrenada UNet

1. Importe de librerías

```
In [3]: import json

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
from pathlib import Path

import os
import zipfile
import shutil
!pip install seaborn
import seaborn as sns
import datetime
!pip install opencv-python
import cv2

import keras
from keras import models
from keras import layers

import random

import tensorflow as tf
```

```
from tensorflow.keras.models import Model
from tensorflow.keras import Layer
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.preprocessing import image
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.preprocessing.image import img_to_array, load_img
from tensorflow.keras.callbacks import ReduceLROnPlateau
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.layers import Conv2D, MaxPooling2D, UpSampling2D, concatenate

!pip install scikit-learn

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score

from PIL import Image # Librería para abrir y manipular imágenes

# ignoring warnings
import warnings
warnings.simplefilter("ignore")
```

Requirement already satisfied: seaborn in /opt/anaconda3/lib/python3.12/site-packages (0.13.2)

Requirement already satisfied: numpy!=1.24.0,>=1.20 in /opt/anaconda3/lib/python3.12/site-packages (from seaborn) (1.26.4)

Requirement already satisfied: pandas>=1.2 in /opt/anaconda3/lib/python3.12/site-packages (from seaborn) (2.2.2)

Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /opt/anaconda3/lib/python3.12/site-packages (from seaborn) (3.9.2)

Requirement already satisfied: contourpy>=1.0.1 in /opt/anaconda3/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.2.0)

Requirement already satisfied: cycler>=0.10 in /opt/anaconda3/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.11.0)

Requirement already satisfied: fonttools>=4.22.0 in /opt/anaconda3/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.51.0)

Requirement already satisfied: kiwisolver>=1.3.1 in /opt/anaconda3/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.4)

Requirement already satisfied: packaging>=20.0 in /opt/anaconda3/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (24.1)

Requirement already satisfied: pillow>=8 in /opt/anaconda3/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (10.4.0)

Requirement already satisfied: pyparsing>=2.3.1 in /opt/anaconda3/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.1.2)

Requirement already satisfied: python-dateutil>=2.7 in /opt/anaconda3/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in /opt/anaconda3/lib/python3.12/site-packages (from pandas>=1.2->seaborn) (2024.1)

Requirement already satisfied: tzdata>=2022.7 in /opt/anaconda3/lib/python3.12/site-packages (from pandas>=1.2->seaborn) (2023.3)

Requirement already satisfied: six>=1.5 in /opt/anaconda3/lib/python3.12/site-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.16.0)

Requirement already satisfied: opencv-python in /opt/anaconda3/lib/python3.12/site-packages (4.11.0.86)

Requirement already satisfied: numpy>=1.21.2 in /opt/anaconda3/lib/python3.12/site-packages (from opencv-python) (1.26.4)

Requirement already satisfied: scikit-learn in /opt/anaconda3/lib/python3.12/site-packages (1.5.1)

Requirement already satisfied: numpy>=1.19.5 in /opt/anaconda3/lib/python3.12/site-packages (from scikit-learn) (1.26.4)

Requirement already satisfied: scipy>=1.6.0 in /opt/anaconda3/lib/python3.12/site-packages (from scikit-learn) (1.13.1)

Requirement already satisfied: joblib>=1.2.0 in /opt/anaconda3/lib/python3.12/site-packages (from scikit-learn) (1.4.2)

Requirement already satisfied: threadpoolctl>=3.1.0 in /opt/anaconda3/lib/python3.12/site-packages (from scikit-learn) (3.5.0)

```
In [4]: #Medir_tiempo_ejecucion():
        inicio = datetime.datetime.now()
        print(f"Inicio: {inicio.strftime('%Y-%m-%d %H:%M:%S.%f')}")
```

Inicio: 2025-05-19 06:26:45.712075

2. Definición de Rutas y Parámetros

```
In [6]: data_dir = 'Datos Finales' # Reemplaza con la ruta real
metadata_file = 'Metadatos_Finales.csv' # Reemplaza con la ruta real
image_height = 128
image_width = 128
batch_size = 32
num_classes = None # Se determinará automáticamente del archivo CSV
```

3. Cargar y procesar los metadatos

```
In [8]: try:
    metadata = pd.read_csv(metadata_file, sep=";")
    metadata = metadata.rename(columns={'Nombre del archivo': 'image_id'})
    metadata = metadata.rename(columns={'Planta_Estado': 'Planta_Estado_Español'})

    # Asegúrate de que las columnas 'filename' (o el nombre de tu columna de
    # y 'label' (o el nombre de tu columna de etiquetas) existan.

    if 'image_id' not in metadata.columns or 'Planta_Estado_Español' not in metadata.columns:
        raise ValueError("El archivo CSV debe contener las columnas 'image_id' y 'Planta_Estado_Español'")
    class_names = metadata['Planta_Estado_Español'].unique().tolist()
    num_classes = len(class_names)
    print(f"Se encontraron {num_classes} clases: {class_names}")
except FileNotFoundError:
    print(f"Error: No se encontró el archivo de metadatos en '{metadata_file}'")
    exit()
except ValueError as e:
    print(f"Error al procesar el archivo CSV: {e}")
    exit()
```

Se encontraron 18 clases: ['Yuca_Enferma', 'Yuca_Saludable', 'Fresa_Saludable', 'Papa_Enferma', 'Maíz_Saludable', 'Tomate_Enferma', 'Manzana_Enferma', 'Maíz_Enferma', 'Tomate_Saludable', 'Manzana_Saludable', 'Fresa_Enferma', 'Papa_Saludable', 'Coliflor_Enferma', 'Pepino_Cohombro_Saludable', 'Fríjol_Enferma', 'Coliflor_Saludable', 'Fríjol_Saludable', 'Pepino_Cohombro_Enferma']

```
In [9]: metadata.Planta_Estado_Español.value_counts()
```

```
Out[9]: Planta_Estado_Español
Yuca_Enferma          18820
Tomate_Enferma        16569
Maíz_Enferma          2690
Yuca_Saludable        2577
Papa_Enferma          2000
Manzana_Saludable     1645
Tomate_Saludable      1591
Manzana_Enferma       1529
Maíz_Saludable        1162
Fresa_Enferma         1109
Fríjol_Enferma        868
Fresa_Saludable       456
Coliflor_Enferma      450
Fríjol_Saludable      428
Pepino_Cohombro_Enferma 350
Pepino_Cohombro_Saludable 341
Coliflor_Saludable    206
Papa_Saludable        152
Name: count, dtype: int64
```

```
In [10]: train, test= train_test_split(metadata, test_size=0.2, random_state=42, shuffle=True)
train.shape, test.shape
```

```
Out[10]: ((42354, 12), (10589, 12))
```

```
In [11]: train.head()
```

Out [11]:

	Dataset	Carpeta	image_i
36463	PlantVillage	Tomato___Tomato_Yellow_Leaf_Curl_Virus	32207a2c-d924-43e7-b466-94832710049d___UF.GRC_
40664	PlantVillage	Tomato___Tomato_Yellow_Leaf_Curl_Virus	87fd49d2-d1b0-4e33-9443-1de82d2c19a1___YLCV_GC
43174	PlantVillage	Tomato___Spider_mites Two-spotted_spider_mite	2fb77f60-ae5b-40c4-b6fb-3c90ac55ccd4___Com.G_S
25436	PlantVillage	Potato___Late_blight	0c2628d4-8d64-48a9-a157-19a9c902e304___RS_LB_4
853	Cassava Leaf Disease Classification	Cassava	1146671365.jp

In [12]: `sample = train.sample(5) # ejemplo común`

In [13]: `print(sample.columns)`

```
Index(['Dataset', 'Carpeta', 'image_id', 'Planta', 'Enfermedad Inglés',
      'Saludable', 'Planta_Estado_Inglés', 'Planta Español',
      'Planta_Estado_Español', 'Nombre traducido por GPT',
      'Nombre Coloquial en Colombia', 'Nombre Coloquial en TV'],
      dtype='object')
```

4. Crear un generador de datos

```
In [15]: # Crear el generador de imágenes
datagen = ImageDataGenerator(
    validation_split = 0.2,
    rescale=1./255
)

train_generator = datagen.flow_from_dataframe(
    dataframe=train,
    directory=data_dir,
    x_col='image_id',
```

```

y_col='Planta_Estado_Español',
target_size=(image_height, image_width),
batch_size=batch_size,
class_mode='categorical', # Importante para clasificación multiclase
subset='training',
seed=42,
classes=class_names # Asegurarse del orden de las clases
)

validation_generator = datagen.flow_from_dataframe(
    dataframe=train,
    directory=data_dir,
    x_col='image_id',
    y_col='Planta_Estado_Español',
    target_size=(image_height, image_width),
    batch_size=batch_size,
    class_mode='categorical',
    subset='validation',
    seed=42,
    classes=class_names
)

```

Found 33880 validated image filenames belonging to 18 classes.

Found 8470 validated image filenames belonging to 18 classes.

```

In [16]: print(train['Planta_Estado_Español'].value_counts())
print(f"Total clases: {train['Planta_Estado_Español'].nunique()}")

```

```

Planta_Estado_Español
Yuca_Enferma          15056
Tomate_Enferma         13255
Maíz_Enferma           2152
Yuca_Saludable         2061
Papa_Enferma           1600
Manzana_Saludable      1316
Tomate_Saludable       1273
Manzana_Enferma        1223
Maíz_Saludable          930
Fresa_Enferma           887
Fríjol_Enferma          694
Fresa_Saludable         365
Coliflor_Enferma        360
Fríjol_Saludable        342
Pepino_Cohombro_Enferma 280
Pepino_Cohombro_Saludable 273
Coliflor_Saludable      165
Papa_Saludable          122
Name: count, dtype: int64
Total clases: 18

```

```

In [17]: from tensorflow.keras.preprocessing import image

```

```

import os
import numpy as np

img_path = os.path.join(data_dir, metadata["image_id"][20])
img = image.load_img(img_path, target_size=(image_height, image_width))
img_tensor = image.img_to_array(img)
img_tensor = np.expand_dims(img_tensor, axis=0)
img_tensor /= 255.

plt.imshow(img_tensor[0])
plt.axis('off')
plt.show()

```



5. Definición del modelo U-Net para clasificación

```

In [19]: epochs=10
         steps_per_epoch = len(train)*0.8 / batch_size
         validation_steps = len(train)*0.2 / batch_size

```

```

In [20]: # DEFINIR EL MODELO U-NET MODIFICADO PARA CLASIFICACIÓN
         def unet_classification_model(input_shape=(image_height, image_width, 3), num
            inputs = Input(shape=input_shape)

            # Encoder
            c1 = Conv2D(32, 3, activation='relu', padding='same')(inputs)

```



```

c1 = Conv2D(32, 3, activation='relu', padding='same')(c1)
p1 = MaxPooling2D()(c1)

c2 = Conv2D(64, 3, activation='relu', padding='same')(p1)
c2 = Conv2D(64, 3, activation='relu', padding='same')(c2)
p2 = MaxPooling2D()(c2)

c3 = Conv2D(128, 3, activation='relu', padding='same')(p2)
c3 = Conv2D(128, 3, activation='relu', padding='same')(c3)
p3 = MaxPooling2D()(c3)

# Bottleneck
c4 = Conv2D(256, 3, activation='relu', padding='same')(p3)
c4 = Conv2D(256, 3, activation='relu', padding='same')(c4)

# Decoder
u5 = UpSampling2D()(c4)
u5 = concatenate([u5, c3])
c5 = Conv2D(128, 3, activation='relu', padding='same')(u5)
c5 = Conv2D(128, 3, activation='relu', padding='same')(c5)

u6 = UpSampling2D()(c5)
u6 = concatenate([u6, c2])
c6 = Conv2D(64, 3, activation='relu', padding='same')(u6)
c6 = Conv2D(64, 3, activation='relu', padding='same')(c6)

u7 = UpSampling2D()(c6)
u7 = concatenate([u7, c1])
c7 = Conv2D(32, 3, activation='relu', padding='same')(u7)
c7 = Conv2D(32, 3, activation='relu', padding='same')(c7)

# Clasificación
gap = GlobalAveragePooling2D()(c7)
outputs = Dense(num_classes, activation='softmax')(gap)

model = Model(inputs=inputs, outputs=outputs)
return model

# CREAR Y COMPILAR EL MODELO
model = unet_classification_model(input_shape=(image_height, image_width, 3))
model.compile(optimizer=Adam(), loss='categorical_crossentropy', metrics=['a
model.summary() # Muestra la arquitectura

```

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 128, 128, 3)	0	–

conv2d (Conv2D)	(None, 128, 128, 32)	896	input_layer[0]
conv2d_1 (Conv2D)	(None, 128, 128, 32)	9,248	conv2d[0][0]
max_pooling2d (MaxPooling2D)	(None, 64, 64, 32)	0	conv2d_1[0][0]
conv2d_2 (Conv2D)	(None, 64, 64, 64)	18,496	max_pooling2d
conv2d_3 (Conv2D)	(None, 64, 64, 64)	36,928	conv2d_2[0][0]
max_pooling2d_1 (MaxPooling2D)	(None, 32, 32, 64)	0	conv2d_3[0][0]
conv2d_4 (Conv2D)	(None, 32, 32, 128)	73,856	max_pooling2d
conv2d_5 (Conv2D)	(None, 32, 32, 128)	147,584	conv2d_4[0][0]
max_pooling2d_2 (MaxPooling2D)	(None, 16, 16, 128)	0	conv2d_5[0][0]
conv2d_6 (Conv2D)	(None, 16, 16, 256)	295,168	max_pooling2d
conv2d_7 (Conv2D)	(None, 16, 16, 256)	590,080	conv2d_6[0][0]
up_sampling2d (UpSampling2D)	(None, 32, 32, 256)	0	conv2d_7[0][0]
concatenate (Concatenate)	(None, 32, 32, 384)	0	up_sampling2d conv2d_5[0][0]
conv2d_8 (Conv2D)	(None, 32, 32, 128)	442,496	concatenate[0]
conv2d_9 (Conv2D)	(None, 32, 32, 128)	147,584	conv2d_8[0][0]
up_sampling2d_1 (UpSampling2D)	(None, 64, 64, 128)	0	conv2d_9[0][0]
concatenate_1 (Concatenate)	(None, 64, 64, 192)	0	up_sampling2d conv2d_3[0][0]
conv2d_10 (Conv2D)	(None, 64, 64,	110,656	concatenate_1

	64)		
conv2d_11 (Conv2D)	(None, 64, 64, 64)	36,928	conv2d_10[0] [
up_sampling2d_2 (UpSampling2D)	(None, 128, 128, 64)	0	conv2d_11[0] [
concatenate_2 (Concatenate)	(None, 128, 128, 96)	0	up_sampling2d. conv2d_1[0] [0
conv2d_12 (Conv2D)	(None, 128, 128, 32)	27,680	concatenate_2
conv2d_13 (Conv2D)	(None, 128, 128, 32)	9,248	conv2d_12[0] [
global_average_pooling2d_2 (GlobalAveragePooling2D)	(None, 32)	0	conv2d_13[0] [
dense (Dense)	(None, 18)	594	global_averag

Total params: 1,947,442 (7.43 MB)

Trainable params: 1,947,442 (7.43 MB)

Non-trainable params: 0 (0.00 B)

```
In [21]: # Earlystopping
from tensorflow.keras.callbacks import EarlyStopping

early_stop = EarlyStopping(
    monitor='val_loss',      # Monitorea la pérdida en validación
    patience=5,              # Número de épocas sin mejora antes de detener
    restore_best_weights=True, # Restaura los mejores pesos
    verbose=1
)
```

6. Entrenamiento del Modelo

```
In [34]: #Entrenamiento
history = model.fit(
    train_generator,
    validation_data=validation_generator,
    epochs=epochs,
    steps_per_epoch=int(steps_per_epoch),
    validation_steps=int(validation_steps),
    callbacks=[early_stop]
)
```

Epoch 1/10
1058/1058 ————— **3010s** 3s/step - accuracy: 0.5344 - loss: 1.6423 - val_accuracy: 0.7392 - val_loss: 0.8817
 Epoch 2/10
1058/1058 ————— **207s** 193ms/step - accuracy: 0.6562 - loss: 1.0248 - val_accuracy: 0.7376 - val_loss: 0.8854
 Epoch 3/10
1058/1058 ————— **3308s** 3s/step - accuracy: 0.7442 - loss: 0.8573 - val_accuracy: 0.7691 - val_loss: 0.7823
 Epoch 4/10
1058/1058 ————— **242s** 226ms/step - accuracy: 0.8125 - loss: 0.7167 - val_accuracy: 0.7706 - val_loss: 0.7798
 Epoch 5/10
1058/1058 ————— **3186s** 3s/step - accuracy: 0.8033 - loss: 0.6215 - val_accuracy: 0.8374 - val_loss: 0.5069
 Epoch 6/10
1058/1058 ————— **203s** 189ms/step - accuracy: 0.9375 - loss: 0.4187 - val_accuracy: 0.8356 - val_loss: 0.5106
 Epoch 7/10
1058/1058 ————— **3230s** 3s/step - accuracy: 0.8490 - loss: 0.4618 - val_accuracy: 0.8615 - val_loss: 0.4100
 Epoch 8/10
1058/1058 ————— **202s** 189ms/step - accuracy: 0.9375 - loss: 0.2299 - val_accuracy: 0.8617 - val_loss: 0.4090
 Epoch 9/10
1058/1058 ————— **2967s** 3s/step - accuracy: 0.8722 - loss: 0.3804 - val_accuracy: 0.8720 - val_loss: 0.4000
 Epoch 10/10
1058/1058 ————— **208s** 195ms/step - accuracy: 0.6562 - loss: 0.8702 - val_accuracy: 0.8741 - val_loss: 0.3872
 Restoring model weights from the end of the best epoch: 10.

```
In [36]: fin = datetime.datetime.now()
print(f"Fin: {fin.strftime('%Y-%m-%d %H:%M:%S.%f')}")

duracion = fin - inicio
print(f"Duración: {duracion}")
```

Fin: 2025-05-19 12:34:26.082779
 Duración: 6:07:40.370704

```
In [38]: import matplotlib.pyplot as plt

# 1. Crear la figura
plt.figure(figsize=(14, 5))

# 2. Gráfico de Accuracy
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Entrenamiento')
plt.plot(history.history['val_accuracy'], label='Validación')
```

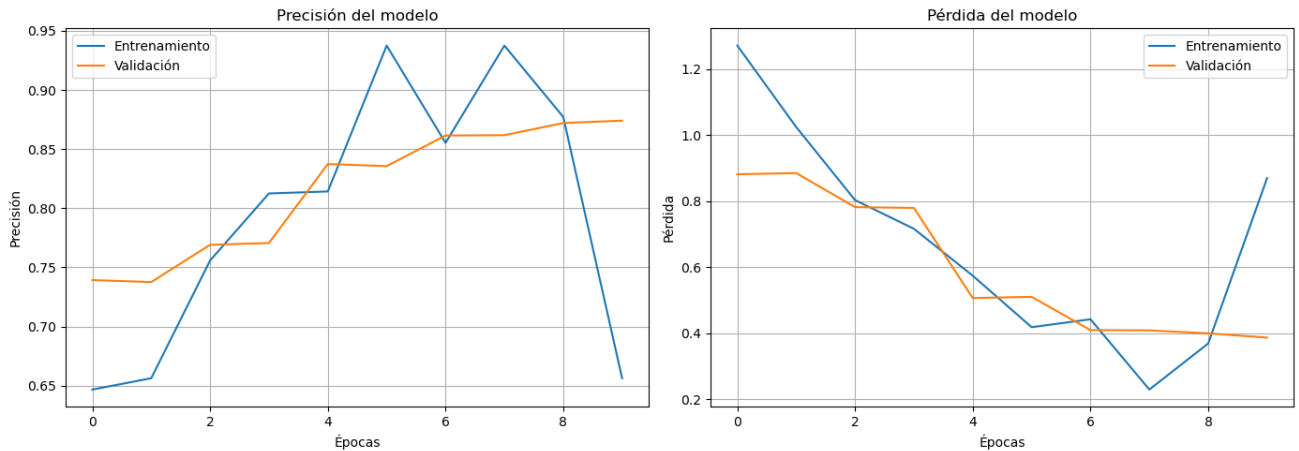
```

plt.title('Precisión del modelo')
plt.xlabel('Épocas')
plt.ylabel('Precisión')
plt.legend()
plt.grid(True)

# 3. Gráfico de Loss
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Entrenamiento')
plt.plot(history.history['val_loss'], label='Validación')
plt.title('Pérdida del modelo')
plt.xlabel('Épocas')
plt.ylabel('Pérdida')
plt.legend()
plt.grid(True)

# 4. Mostrar
plt.tight_layout()
plt.show()

```



```
In [40]: print(test.columns.tolist())
```

```

['Dataset', 'Carpeta', 'image_id', 'Planta', 'Enfermedad Inglés', 'Saludabl
e', 'Planta_Estado_Inglés', 'Planta Español', 'Planta_Estado_Español', 'Nomb
re traducido por GPT', 'Nombre Coloquial en Colombia', 'Nombre Coloquial en
TV']

```

```
In [42]: epochs = list(range(1, 11))
```

```

loss = history.history['loss']
val_loss = history.history['val_loss']
accuracy = history.history['accuracy']
val_accuracy = history.history['val_accuracy']

data = {
    'Epoch': epochs,

```

```

    'Trainning Loss': loss,
    'Validation Loss': val_loss,
    'Training Accuracy': accuracy,
    'Validation Accuracy': val_accuracy
}
df = pd.DataFrame(data)

# Imprimir la tabla
print(df)

```

	Epoch	Trainning Loss	Validation Loss	Training Accuracy \
0	1	1.271352	0.881734	0.646685
1	2	1.024798	0.885447	0.656250
2	3	0.803489	0.782273	0.755791
3	4	0.716667	0.779848	0.812500
4	5	0.574587	0.506937	0.814169
5	6	0.418700	0.510574	0.937500
6	7	0.442693	0.409960	0.855265
7	8	0.229851	0.408964	0.937500
8	9	0.369363	0.400015	0.877186
9	10	0.870238	0.387152	0.656250

	Validation Accuracy
0	0.739228
1	0.737571
2	0.769058
3	0.770597
4	0.837358
5	0.835582
6	0.861506
7	0.861742
8	0.872041
9	0.874053

```

In [44]: print('Perdida final en Train:', history.history['loss'][-1])
print("Pérdida final en Validation:", history.history['val_loss'][-1])

print("Accuracy final en Train:", history.history['accuracy'][-1])
print("Accuracy final en Validation:", history.history['val_accuracy'][-1])

```

Perdida final en Train: 0.8702376484870911
 Pérdida final en Validation: 0.38715216517448425
 Accuracy final en Train: 0.65625
 Accuracy final en Validation: 0.8740530014038086

```

In [46]: #validación con datos de test

```

```

test_generator = datagen.flow_from_dataframe(test,
    directory = "Datos Finales",
    subset = "training",
    x_col = "image_id",

```

```
y_col = "Planta_Estado_Español",
target_size = (image_height, image_width),
batch_size = batch_size,
class_mode = "categorical")
```

Found 8472 validated image filenames belonging to 18 classes.

7. Testeo del Modelo

```
In [49]: # Evaluar sobre el set de test
test_loss, test_accuracy = model.evaluate(test_generator, steps=len(test_gen

print(f'🔍 Pérdida en test: {test_loss:.4f}')
print(f'✅ Precisión en test: {test_accuracy:.2%}')
```

265/265 ————— 178s 672ms/step – accuracy: 0.0267 – loss: 11.3917

🔍 Pérdida en test: 11.2555

✅ Precisión en test: 2.77%

```
In [50]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns

# 1. Predecir clases sobre el test set
# Obtener las predicciones del modelo
y_pred_probs = model.predict(test_generator, steps=len(test_generator), vert

# Convertir probabilidades a etiquetas (argmax)
y_pred = np.argmax(y_pred_probs, axis=1)

# 2. Obtener etiquetas verdaderas
# Las verdaderas etiquetas están en test_generator.classes
y_true = test_generator.classes

# 3. Obtener los nombres de las clases
class_labels = list(test_generator.class_indices.keys())

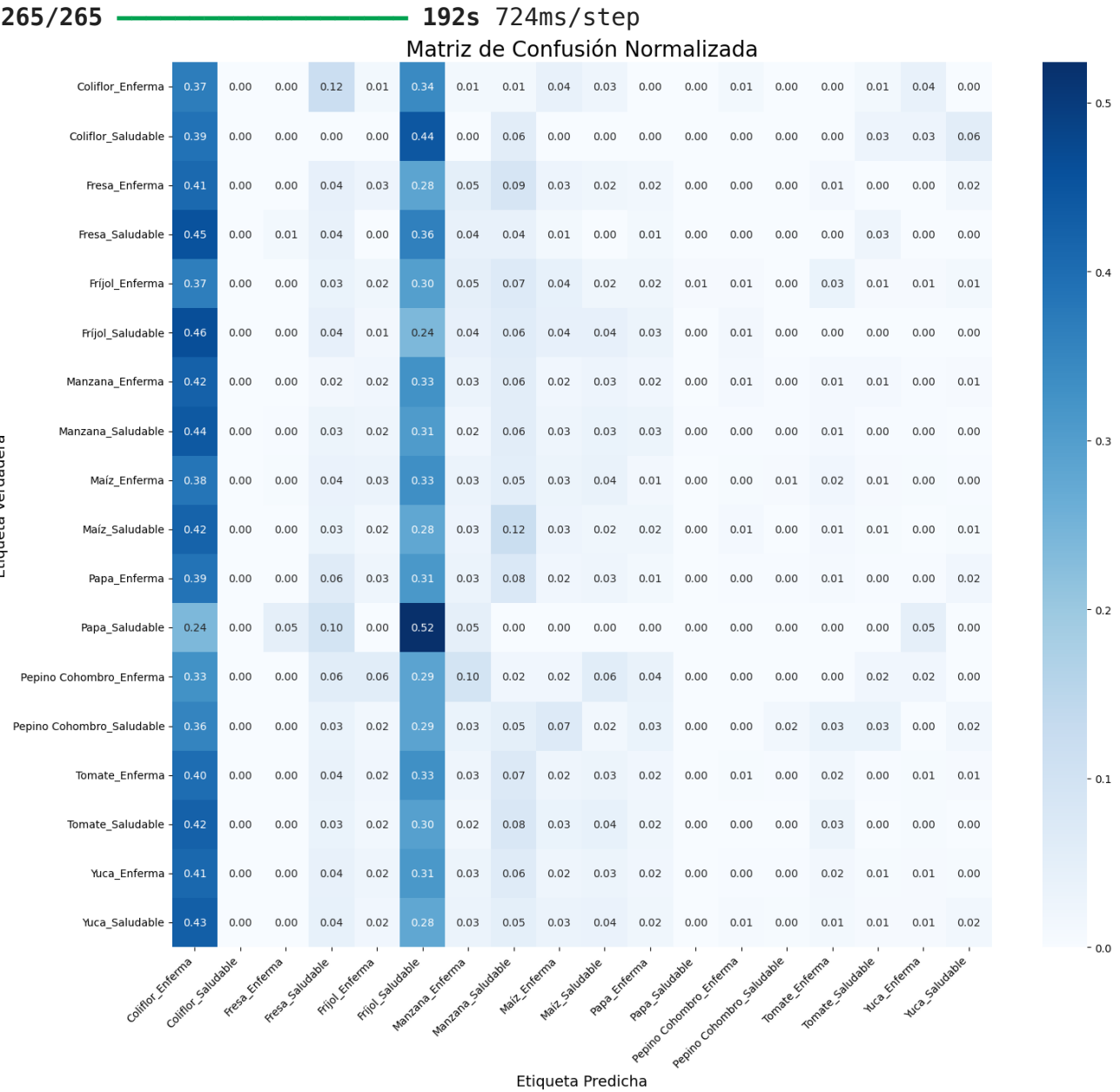
# 4. Calcular la matriz de confusión
cm = confusion_matrix(y_true, y_pred)

# 5. Normalizar la matriz de confusión (opcional para mejor visualización)
cm_normalized = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]

# 6. Graficar la matriz de confusión
plt.figure(figsize=(16, 14))
sns.heatmap(cm_normalized, annot=True, fmt='.2f', cmap='Blues', xticklabels=
```

```
plt.title('Matriz de Confusión Normalizada', fontsize=20)
plt.ylabel('Etiqueta Verdadera', fontsize=14)
plt.xlabel('Etiqueta Predicha', fontsize=14)
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

# 7. Reporte de clasificación (precisión, recall, f1-score por clase)
print('\nReporte de Clasificación:')
print(classification_report(y_true, y_pred, target_names=class_labels, digit
```

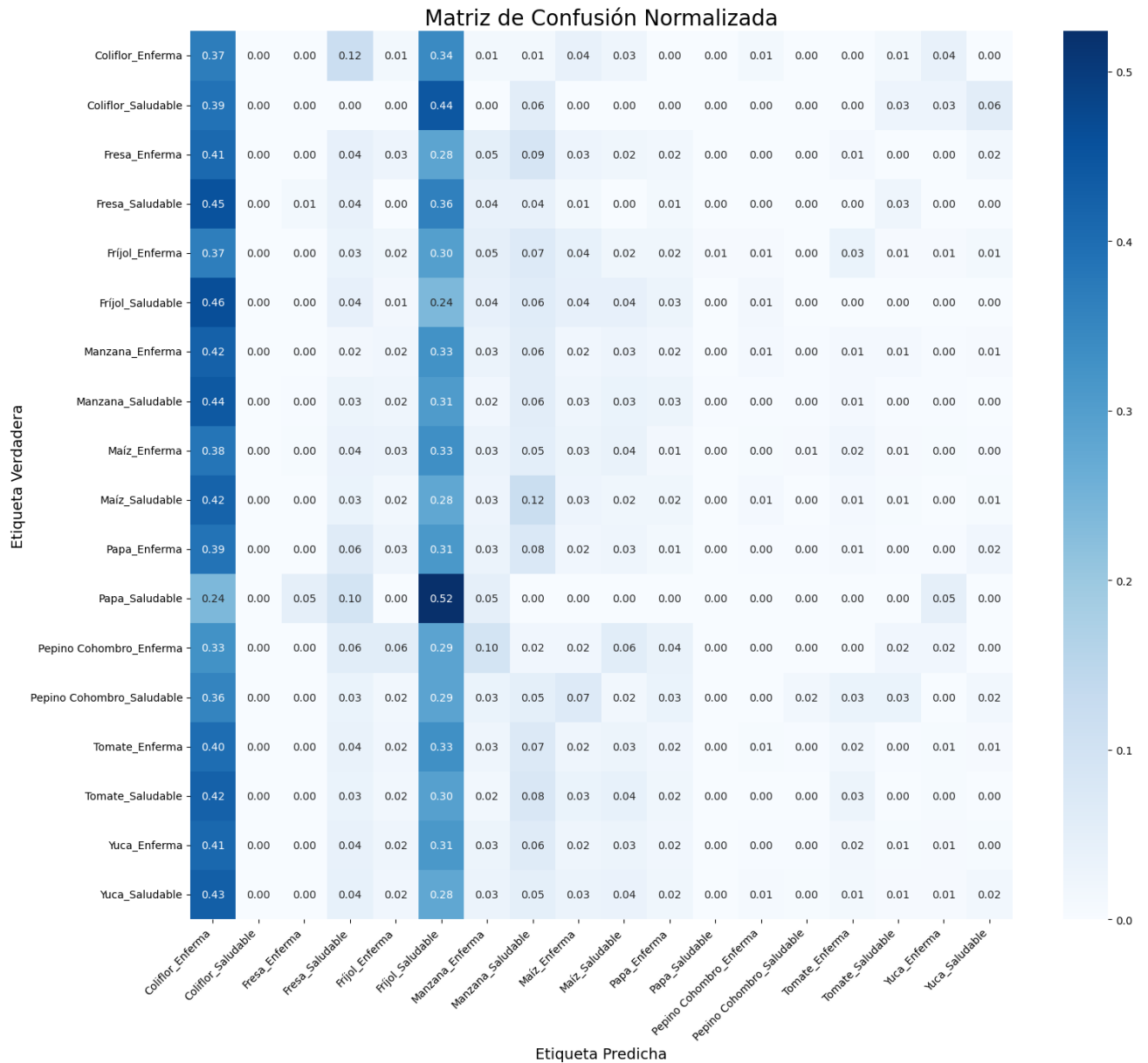


Reporte de Clasificación:

	precision	recall	f1-score	support
Coliflor_Enferma	0.0081	0.3684	0.0159	76
Coliflor_Saludable	0.0000	0.0000	0.0000	36
Fresa_Enferma	0.0000	0.0000	0.0000	179
Fresa_Saludable	0.0090	0.0390	0.0146	77
Fríjol_Enferma	0.0164	0.0208	0.0183	144
Fríjol_Saludable	0.0059	0.2388	0.0116	67
Manzana_Enferma	0.0325	0.0327	0.0326	245
Manzana_Saludable	0.0266	0.0566	0.0362	265
Maíz_Enferma	0.0553	0.0288	0.0379	417
Maíz_Saludable	0.0161	0.0219	0.0185	183
Papa_Enferma	0.0241	0.0128	0.0167	312
Papa_Saludable	0.0000	0.0000	0.0000	21
Pepino Cohombro_Enferma	0.0000	0.0000	0.0000	52
Pepino Cohombro_Saludable	0.0357	0.0169	0.0230	59
Tomate_Enferma	0.3529	0.0179	0.0341	2676
Tomate_Saludable	0.0000	0.0000	0.0000	236
Yuca_Enferma	0.4259	0.0076	0.0150	3008
Yuca_Saludable	0.1228	0.0167	0.0294	419
accuracy			0.0203	8472
macro avg	0.0629	0.0488	0.0169	8472
weighted avg	0.2752	0.0203	0.0234	8472

```
In [51]: # 1. Guardar la figura de la matriz de confusión
plt.figure(figsize=(16, 14))
sns.heatmap(cm_normalized, annot=True, fmt='.2f', cmap='Blues', xticklabels=
plt.title('Matriz de Confusión Normalizada', fontsize=20)
plt.ylabel('Etiqueta Verdadera', fontsize=14)
plt.xlabel('Etiqueta Predicha', fontsize=14)
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()

# Guardar el gráfico como imagen
plt.savefig(r'C:\Proyecto de grado\Modelos\matriz_confusion.png', dpi=300)
plt.show()
```



```
In [52]: # 2. Guardar el reporte de clasificación en CSV

# Convertir el reporte a un diccionario
from sklearn.metrics import classification_report

report = classification_report(y_true, y_pred, target_names=class_labels, out

# Convertirlo a DataFrame
report_df = pd.DataFrame(report).transpose()

# Guardar como CSV
report_df.to_csv(r'C:\Proyecto de grado\Modelos\reporte_clasificacion.csv',

print("✅ Reporte de clasificación guardado en C:\\Proyecto de grado\\Modelos\\reporte_clasificacion.csv")
```

✓ Reporte de clasificación guardado en C:\Proyecto de grado\Modelos\reporte_clasificacion.csv

```
In [53]: accuracy = accuracy_score(y_true, y_pred)
         print(f"Test Accuracy: {accuracy:.4f}")
```

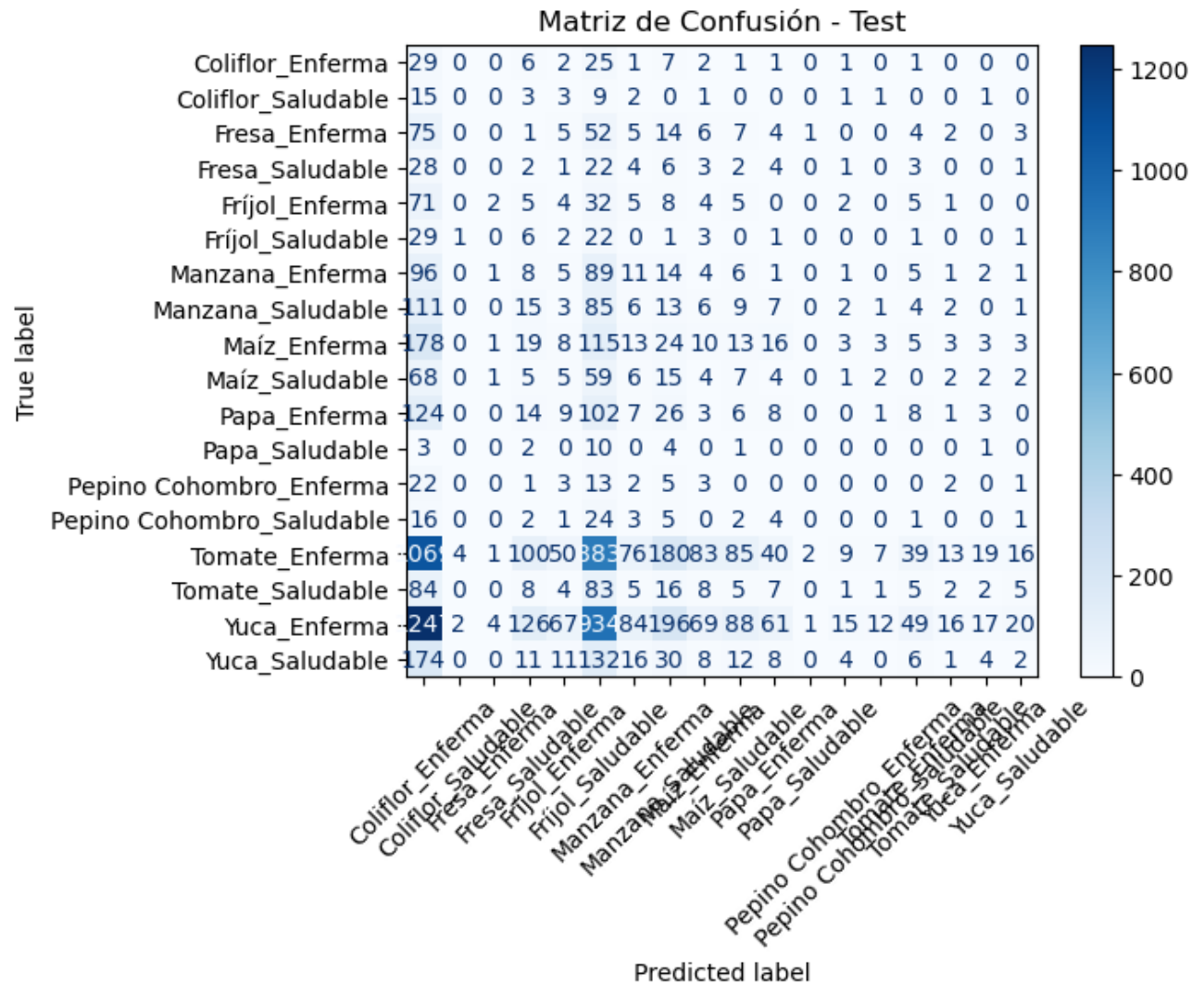
Test Accuracy: 0.0203

```
In [54]: # -----
         # 1. Obtener predicciones completas para test
         # -----
         y_pred_probs = model.predict(test_generator)
         y_pred = np.argmax(y_pred_probs, axis=1)
         y_true = test_generator.classes
```

265/265 ————— 199s 752ms/step

```
In [55]: # -----
         # 2. Matriz de Confusión
         # -----
         from sklearn.metrics import confusion_matrix, accuracy_score, ConfusionMatrixDisplay

         cm = confusion_matrix(y_true, y_pred)
         disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=test_generator.classes)
         disp.plot(cmap="Blues", xticks_rotation=45)
         plt.title("Matriz de Confusión - Test")
         plt.show()
```



```
In [56]: # Función para normalizar imágenes solo para visualización
def normalize_for_display(img):
    img_min = img.min()
    img_max = img.max()
    if img_max > 1.0 or img_min < 0.0:
        img = (img - img_min) / (img_max - img_min + 1e-7)
    return img

# Obtener nombres de clases
class_indices = test_generator.class_indices
class_names = list(class_indices.keys())

# Reiniciar el generador para empezar desde el principio
test_generator.reset()
x_batch, y_true_batch = next(test_generator)

# Obtener predicciones del modelo
y_pred_probs_batch = model.predict(x_batch)
y_pred_batch = np.argmax(y_pred_probs_batch, axis=1)
```

```
y_true_batch = np.argmax(y_true_batch, axis=1)

# Número de imágenes a mostrar (máximo el tamaño del batch)
num_samples = x_batch.shape[0]

# Mostrar imágenes con etiquetas
plt.figure(figsize=(15, 8))
for i in range(num_samples):
    plt.subplot(3, 4, i + 1)
    img_to_show = normalize_for_display(x_batch[i])
    plt.imshow(img_to_show)
    plt.axis('off')
    plt.title(
        f"Real: {class_names[y_true_batch[i]]}\nPred: {class_names[y_pred_batch[i]]}\n"
        f"color='green' if y_true_batch[i] == y_pred_batch[i] else 'red'"
    )
plt.tight_layout()
plt.suptitle("Muestreo de Predicciones vs. Etiquetas Reales", fontsize=16, y=0.95)
plt.show()
```

1/1 ————— 1s 823ms/step

```

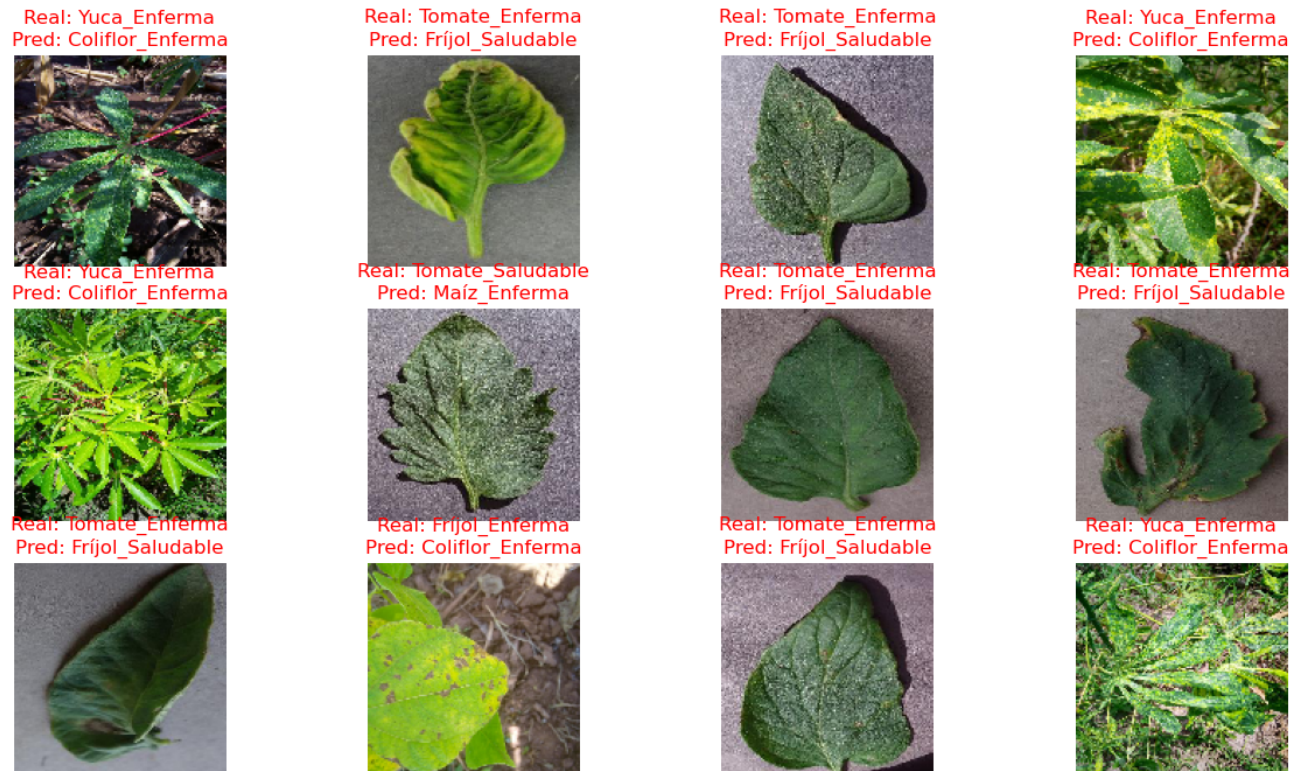
-----
ValueError                                Traceback (most recent call last)
Cell In[56], line 28
    26 plt.figure(figsize=(15, 8))
    27 for i in range(num_samples):
----> 28     plt.subplot(3, 4, i + 1)
    29     img_to_show = normalize_for_display(x_batch[i])
    30     plt.imshow(img_to_show)

File /opt/anaconda3/lib/python3.12/site-packages/matplotlib/pyplot.py:1534,
in subplot(*args, **kwargs)
    1531 fig = gcf()
    1533 # First, search for an existing subplot with a matching spec.
-> 1534 key = SubplotSpec._from_subplot_args(fig, args)
    1536 for ax in fig.axes:
    1537     # If we found an Axes at the position, we can reuse it if the us
er passed no
    1538     # kwargs or if the Axes class and kwargs are identical.
    1539     if (ax.get_subplotspec() == key
    1540         and (kwargs == {}
    1541             or (ax._projection_init
    1542                 == fig._process_projection_requirements(**kwargs)))
    ):

File /opt/anaconda3/lib/python3.12/site-packages/matplotlib/gridspec.py:589,
in SubplotSpec._from_subplot_args(figure, args)
    587 else:
    588     if not isinstance(num, Integral) or num < 1 or num > rows*cols:
--> 589         raise ValueError(
    590             f"num must be an integer with 1 <= num <= {rows*cols}, "
    591             f"not {num!r}"
    592         )
    593     i = j = num
    594 return gs[i-1:j]

ValueError: num must be an integer with 1 <= num <= 12, not 13

```



```
In [64]: from sklearn.metrics import roc_curve, auc
from tensorflow.keras.utils import to_categorical

# 3. Convertir etiquetas reales a one-hot
n_classes = y_pred_probs.shape[1]
y_true_onehot = to_categorical(y_true, num_classes=n_classes)

# 4. Calcular ROC y AUC por clase
fpr = dict()
tpr = dict()
roc_auc = dict()

for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_true_onehot[:, i], y_pred_probs[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

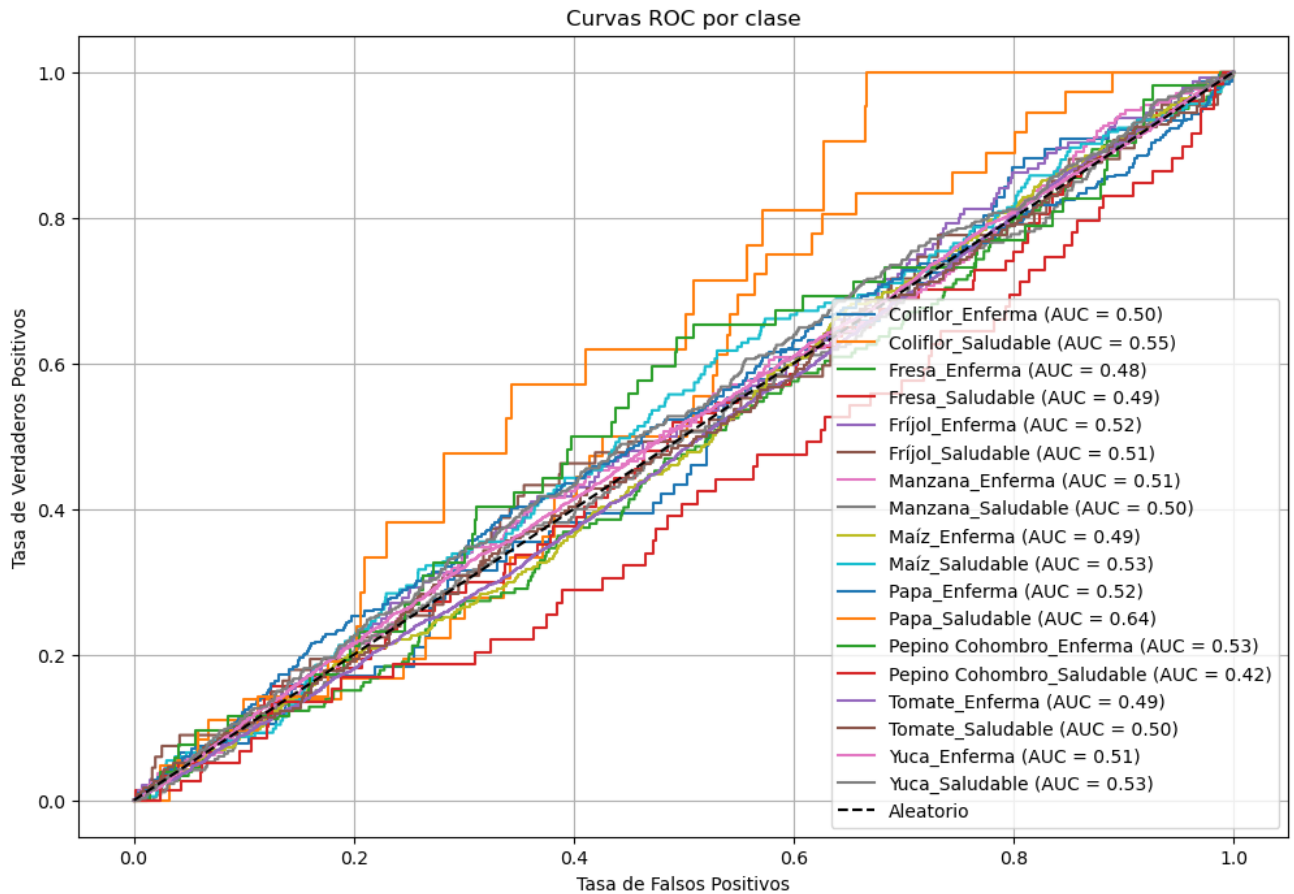
# 5. Obtener nombres de clase
class_names = list(test_generator.class_indices.keys())

# 6. Graficar curvas ROC
plt.figure(figsize=(12, 8))
for i in range(n_classes):
    plt.plot(fpr[i], tpr[i], label=f'{class_names[i]} (AUC = {roc_auc[i]:.2f})')

plt.plot([0, 1], [0, 1], 'k--', label='Aleatorio')
plt.xlabel('Tasa de Falsos Positivos')
plt.ylabel('Tasa de Verdaderos Positivos')
```



```
plt.title('Curvas ROC por clase')
plt.legend(loc='lower right')
plt.grid(True)
plt.show()
```



```
In [65]: from sklearn.metrics import roc_auc_score

# AUC macro (promedio de clases, todas igual de importantes)
auc_macro = roc_auc_score(y_true_onehot, y_pred_probs, average='macro')

# AUC micro (promedia sobre todas las instancias)
auc_micro = roc_auc_score(y_true_onehot, y_pred_probs, average='micro')

print(f"AUC Macro promedio: {auc_macro:.4f}")
print(f"AUC Micro promedio: {auc_micro:.4f}")
```

AUC Macro promedio: 0.5114

AUC Micro promedio: 0.5467

```
In [68]: # Asegúrate de tener esto:
# y_pred = np.argmax(y_pred_probs, axis=1)
# y_true = test_generator.classes

# Accuracy
accuracy = accuracy_score(y_true, y_pred)
```



```

# F1 Scores
f1_macro = f1_score(y_true, y_pred, average='macro')
f1_micro = f1_score(y_true, y_pred, average='micro')
f1_weighted = f1_score(y_true, y_pred, average='weighted')

print(f"Accuracy: {accuracy:.4f}")
print(f"F1 Score (macro): {f1_macro:.4f}")
print(f"F1 Score (micro): {f1_micro:.4f}")
print(f"F1 Score (weighted): {f1_weighted:.4f}")

# Detalle por clase
class_names = list(test_generator.class_indices.keys())
print("\nReporte de clasificación por clase:")
print(classification_report(y_true, y_pred, target_names=class_names))

```

Accuracy: 0.0196
 F1 Score (macro): 0.0168
 F1 Score (micro): 0.0196
 F1 Score (weighted): 0.0201

Reporte de clasificación por clase:

	precision	recall	f1-score	support
Coliflor_Enferma	0.01	0.38	0.02	76
Coliflor_Saludable	0.00	0.00	0.00	36
Fresa_Enferma	0.00	0.00	0.00	179
Fresa_Saludable	0.01	0.03	0.01	77
Fríjol_Enferma	0.02	0.03	0.02	144
Fríjol_Saludable	0.01	0.33	0.02	67
Manzana_Enferma	0.04	0.04	0.04	245
Manzana_Saludable	0.02	0.05	0.03	265
Maíz_Enferma	0.05	0.02	0.03	417
Maíz_Saludable	0.03	0.04	0.03	183
Papa_Enferma	0.05	0.03	0.03	312
Papa_Saludable	0.00	0.00	0.00	21
Pepino Cohombro_Enferma	0.00	0.00	0.00	52
Pepino Cohombro_Saludable	0.00	0.00	0.00	59
Tomate_Enferma	0.29	0.01	0.03	2676
Tomate_Saludable	0.04	0.01	0.01	236
Yuca_Enferma	0.31	0.01	0.01	3008
Yuca_Saludable	0.04	0.00	0.01	419
accuracy			0.02	8472
macro avg	0.05	0.05	0.02	8472
weighted avg	0.21	0.02	0.02	8472

8. Guardar el modelo entrenado

```
In [71]: model.save('modelo_16_Unet_PlantaEstado.h5')
```

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

```
In [73]: fin = datetime.datetime.now()
print(f"Fin: {fin.strftime('%Y-%m-%d %H:%M:%S.%f')}")

duracion = fin - inicio
print(f"Duración: {duracion}")
```

Fin: 2025-05-19 12:45:15.446195

Duración: 6:18:29.734120

```
In [75]: class_indices = train_generator.class_indices # o test_generator.class_indices
index_to_class = {v: k for k, v in class_indices.items()}
print(index_to_class)
with open("index_to_class_modelo16_NoBalanceado.json", "w") as f:
    json.dump(index_to_class, f, indent=4, ensure_ascii=False)
```

```
{0: 'Yuca_Enferma', 1: 'Yuca_Saludable', 2: 'Fresa_Saludable', 3: 'Papa_Enferma', 4: 'Maíz_Saludable', 5: 'Tomate_Enferma', 6: 'Manzana_Enferma', 7: 'Maíz_Enferma', 8: 'Tomate_Saludable', 9: 'Manzana_Saludable', 10: 'Fresa_Enferma', 11: 'Papa_Saludable', 12: 'Coliflor_Enferma', 13: 'Pepino Cohombro_Saludable', 14: 'Fríjol_Enferma', 15: 'Coliflor_Saludable', 16: 'Fríjol_Saludable', 17: 'Pepino Cohombro_Enferma'}
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```